UNIVERSIDAD NACIONAL DE SAN AGUSTÍN

FACULTAD DE INGENIERÍA DE PROCESOS Y SERVICIOS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



Tecnología de Objetos

Informe de avance de proyecto SIGER

Docente

APAZA ACEITUNO RONI GUILLERMO

Estudiantes

Mejia Rondan Giovanni Patrick

Bacca Calsin Leonardo Juan José

Chipana Jerónimo German Arturo

Ponce Llerena Renato Xavier

Arequipa-Perú

2026

Índice

1. Introducción y Resumen Ejecutivo

2. Objetivos del Proyecto

- 2.1. Objetivo General
- 2.2. Objetivos Específicos

3. Marco Teórico y Aplicación de Temas de Tecnología de Objetos

- 3.1. Programación Orientada a Objetos Avanzada (POO)
- 3.2. Programación Genérica (Templates) y Gestión de Memoria STL
- 3.3. Programación Concurrente y MultiThreading
- 3.4. Patrones de Diseño de Software (GoF)
- 3.5. Arquitectura Modelo - Vista - Controlador (MVC)

4. Arquitectura y Componentes del Sistema SIGER

5. Mecanismo de Simulación y Lógica de Flota

- 5.1. Composición de la Flota Logística
- 5.2. Asignación Exclusiva por Tipo y Espera Reactiva
- 5.3. Tiempos de Recorrido Calibrados y Animación Sincronizada
- 5.4. Finalización Automática de Simulación

6. Mapa Metropolitano de Arequipa y Georreferenciación

- 6.1. Integración Cartográfica en Alta Definición
- 6.2. Proyección Matemática Web Mercator
- 6.3. Catálogo de los 20 Nodos Logísticos Metropolitanos
- 6.4. Navegación e Interacción Tipo Google Maps

7. Calidad de Software y Verificación (Unit Testing)

8. Trazabilidad de Trabajo y Control de Versiones (Git / GitHub)

9. Conclusiones y Trabajo Futuro

- 9.1. Conclusiones
- 9.2. Trabajo Futuro y Líneas de Expansión

1. Introducción y Resumen Ejecutivo

SIGER (Sistema Integrado de Gestión y Simulación de Entregas Rápidas) es una aplicación de escritorio en C++17 y Qt 6 que simula la logística de última milla en Arequipa. El usuario registra pedidos, una flota de cinco repartidores los atiende en paralelo (un hilo por repartidor) y todo el proceso se monitorea en tiempo real sobre un mapa de la ciudad. Es el proyecto final del curso de Tecnología de Objetos de la UNSA.

Este informe resume el estado del proyecto al 6 de noviembre de 2026, según el contenido del repositorio (https://github.com/ChipanaGerman/SIGER). El sistema ya funciona de punta a punta: los seis temas exigidos por el curso están implementados, las 75 pruebas unitarias pasan y el código suma unas 5 000 líneas repartidas en 14 commits.

Tema del curso	Cómo se aplica en SIGER	Estado
POO avanzada	Clases abstractas `Vehiculo` y `Paquete`, polimorfismo, destructores virtuales	Completo
Genéricos y STL	`Cola<T>`, `Repositorio<T, Key>`, `Ruta<CoordType>`, functores, smart pointers	Completo
MultiThreading	`std::thread` por repartidor, `mutex`, `condition_variable`, `atomic`	Completo
Patrones GoF	Factory Method (flota) y Observer (modelo → interfaz)	Completo
MVC	Carpetas `model/`, `controller/` y `view/` con dependencias en un solo sentido	Completo
GUI en Qt 6	Dashboard, mapa 2D, estadísticas con QtCharts, panel de control, formulario	Completo

2. Objetivos del Proyecto

2.1. Objetivo General

Diseñar e implementar un sistema de simulación y monitoreo de entregas urbanas en tiempo real, aplicando POO, programación genérica, concurrencia, patrones de diseño GoF y arquitectura MVC sobre C++17 y Qt 6.

2.2. Objetivos Específicos

- **Dominio:** modelar vehículos (moto, auto, camión) y paquetes (estándar, frágil) con costos, velocidades y prioridades distintas.
- **Genéricos:** construir estructuras reutilizables y seguras ante concurrencia, con gestión de memoria mediante punteros inteligentes.
- **Concurrencia:** ejecutar cada repartidor en su propio hilo sin condiciones de carrera ni interbloqueos.
- **Patrones y arquitectura:** desacoplar la creación de vehículos (Factory Method) y la comunicación modelo–interfaz (Observer) dentro de un esquema MVC.
- **Interfaz:** ofrecer un panel interactivo con tabla de pedidos, mapa de Arequipa, estadísticas y controles de simulación.

- **Calidad:** respaldar el comportamiento con pruebas unitarias automatizadas integradas en CMake/CTest.

3. Marco Teórico y Aplicación de Temas de Tecnología de Objetos

3.1. Programación Orientada a Objetos Avanzada (POO)

El dominio se apoya en dos jerarquías abstractas. `Vehiculo` define tres métodos virtuales puros (`calcularCosto`, `getTipo`, `calcularTiempoEntrega`) y dos virtuales con implementación base (`puedeCargar`, `getDescripcion`). `Paquete` exige `calcularPrioridad`, `getTipo` y `requiereManejoEspecial`. Ambas declaran destructor virtual, de modo que borrar un objeto derivado a través de un puntero base es seguro. Los atributos son `protected` o `private` y se exponen con getters; los cambios de estado pasan por setters.

```
class Vehiculo {                        // src/model/Vehiculo.h (extracto)
public:
    virtual ~Vehiculo() = default;
    virtual double calcularCosto(double distanciaKm, double pesoKg) const = 0;
    virtual std::string getTipo() const = 0;
    virtual double calcularTiempoEntrega(double distanciaKm) const = 0;
    virtual std::string getDescripcion() const;
    virtual bool puedeCargar(double pesoKg) const;
};
```

Cada subclase resuelve el cálculo a su manera, lo que da el polimorfismo en tiempo de ejecución:

Vehículo	Velocidad	Costo de entrega (S/.)
`Moto`	60 km/h	0.80 por km + 0.50 por cada kg que exceda 5 kg
`Auto`	45 km/h	1.50 por km + 0.30 por kg
`Camion`	30 km/h	5.00 fijos + 2.50 por km + 0.10 por kg

Los paquetes se diferencian por prioridad (escala de 1 a 10). `PaqueteFragil` parte de 7, suma 2 si el valor asegurado supera S/. 1 000 y 1 más si pesa menos de 2 kg, con tope en 10; además siempre requiere manejo especial.

3.2. Programación Genérica (Templates) y Gestión de Memoria STL

- **Cola<T>:** cola FIFO protegida con `std::mutex`. Incluye `extraerSi(pred)` y `contieneSi(pred)`, que reciben un predicado y permiten sacar el primer pedido compatible con un vehículo sin alterar el orden de los demás (devuelve `std::optional<T>`).
- **Repositorio<T, Key = std::string>:** almacén de entidades indexado por clave, con búsquedas filtradas por predicados (lambdas).
- **Ruta<CoordType = double>:** secuencia de puntos con cálculo de distancia euclidiana, parametrizada por el tipo de coordenada.

- **Functores (Estrategias.h):** `AsignarPorPrioridad`, `FiltrarPorEstado` y `AsignarRepartidorOptimo`, estructuras con `operator()`.
- **Punteros inteligentes:** `shared_ptr` para pedidos y repartidores compartidos entre cola, repositorio e hilos; `weak_ptr` para la lista de observadores, lo que evita ciclos de referencia. No hay `new`/`delete` manual en el modelo.

3.3. Programación Concurrente y MultiThreading

`SimuladorEntregas` lanza un `std::thread` por repartidor (`hiloRepartidor`). Los hilos comparten una cola de pedidos y se coordinan así:

- **Espera reactiva:** el hilo duerme en `condition_variable::wait_for` (200 ms) y solo se despierta si hay un pedido compatible con su tipo de vehículo y su capacidad. Registrar un pedido o liberar un repartidor ejecuta `notify_all()`.
- **Atómicos:** los indicadores `m_corriendo` y `m_pausado` y los contadores de entregas en camino y fallidas son `std::atomic`, por lo que se leen sin bloquear.
- **Despacho escalonado:** un `m_despachoMutex` separado obliga a que las salidas estén al menos 1 200 ms una de otra, para que no partan todos en el mismo instante.
- **Cierre sin interbloqueo:** `detener()` mueve los hilos a un vector local dentro del lock y hace `join()` fuera de él.
- **GeneradorEventos:** hilo que crea pedidos aleatorios cada 2 500 ms. Está implementado, pero en la versión actual la llamada a `iniciar()` está comentada para que la simulación procese solo los pedidos registrados a mano.

3.4. Patrones de Diseño de Software (GoF)

A. Patrón Creacional: Factory Method

`VehiculoFactory` define la creación y sus subclases `MotoFactory`, `AutoFactory` y `CamionFactory` producen cada tipo. `FactoriaVehiculos::crear("Moto", id, placa, capacidad)` mantiene un registro de creadores y devuelve un `shared_ptr<Vehiculo>`, así `main.cpp` arma la flota sin conocer las clases concretas.

B. Patrón de Comportamiento: Observer

`GestorPedidos` y `SimuladorEntregas` son sujetos (`ISubject`) y notifican diez eventos tipados (`EventoSiger`), entre ellos `PEDIDO_NUEVO`, `PEDIDO_EN_CAMINO`, `PEDIDO_COMPLETADO`, `SIMULACION_PAUSADA` y `TODOS_COMPLETADOS`, junto con un `DatoEvento`. Los controladores implementan `IObserver::onEvento` y convierten cada evento en una señal de Qt, de modo que los hilos del modelo nunca tocan la interfaz directamente.

3.5. Arquitectura Modelo - Vista - Controlador (MVC)

La regla es que el **modelo** no depende de Qt: es C++ estándar y por eso las pruebas unitarias lo compilan solo. Los **controladores** (`QObject` + `IObserver`) traducen las acciones de la

vista a llamadas al modelo y los eventos del modelo a señales. Las **vistas** solo conocen señales y slots de los controladores.

4. Arquitectura y Componentes del Sistema SIGER

El proyecto se compila con CMake (C++17, Qt6 `Core`, `Widgets` y `Charts`) y genera cuatro ejecutables: la aplicación `SIGER` y las pruebas `test_dominio`, `test_generico` y `test_threading`. El mapa va empaquetado como recurso Qt (`resources.qrc`).

Capa	Archivos principales	Responsabilidad	Líneas
Modelo	`Vehiculo`, `Moto`, `Auto`, `Camion`, `Paquete*`, `Pedido`, `Repartidor`	Entidades de dominio y cálculo de costos	
	`Cola.h`, `Repositorio.h`, `Ruta`, `Estrategias.h`	Genéricos y functores	
	`SimuladorEntregas`, `GestorPedidos`, `GeneradorEventos`	Hilos, sincronización y estadísticas	
	`VehiculoFactory`, `Observer.h`	Factory Method y Observer	2 769
Controlador	`PedidoController`, `SimulacionController`	Mediación entre eventos del modelo y señales Qt	361
Vista	`MainWindow`, `DashboardView`, `MapaView`, `EstadisticasView`, `ControlPanel`, `FormPedido`	Interfaz gráfica	1 309
Otros	`main.cpp`, `tests/*.cpp`	Ensamblado de la flota y pruebas	77 + 493

5. Mecanismo de Simulación y Lógica de Flota

5.1. Composición de la Flota Logística

`main.cpp` crea cinco repartidores con `FactoriaVehiculos::crear`:

Repartidor	Vehículo	Placa	Capacidad
R-101 Carlos Quispe	Moto (V-01)	MTO-501	15 kg
R-102 Lucía Torres	Moto (V-02)	MTO-502	15 kg
R-103 María Flores	Auto (V-03)	AUT-701	100 kg
R-104 Jorge Ramos	Auto (V-04)	AUT-702	100 kg
R-105 Pedro Mamani	Camión (V-05)	CAM-901	2 000 kg

5.2. Asignación Exclusiva por Tipo y Espera Reactiva

Cada pedido puede traer un tipo de vehículo requerido, elegido en el formulario. `Pedido::esCompatibleCon(tipo, capacidad)` rechaza el pedido si el paquete excede la capacidad y, cuando hay un tipo exigido, solo acepta ese tipo. Por ejemplo, con tres pedidos de moto y solo dos motos, el tercero queda en la cola; ni los autos ni el camión lo toman.

Apenas una moto termina y queda libre, el `notify_all()` la despierta y toma el pedido pendiente.

5.3. Tiempos de Recorrido Calibrados y Animación Sincronizada

Cada viaje dura un tiempo base según el tipo de vehículo: **5.5 s** para moto, **7.5 s** para auto y **10 s** para camión. Ese tiempo se multiplica por el valor del control de velocidad de la interfaz (100 a 3 000 ms por paso; 1 000 por defecto). El hilo avanza en pasos de 50 ms y respeta la pausa. Por debajo de 100 ms la simulación entra en modo de prueba (sin escalonamiento y con viajes de 10 ms), que es el que usan las pruebas de concurrencia. El mapa anima el móvil con el mismo evento `PEDIDO_EN_CAMINO`, por lo que el recorrido visual y el hilo van sincronizados.

5.4. Finalización Automática de Simulación

Después de cada pedido completado o cancelado, `GestorPedidos` compara completados + cancelados con el total registrado. Si coinciden, emite `TODOS_COMPLETADOS`; `SimulacionController` detiene los hilos y la ventana principal avisa que el lote terminó.

6. Mapa Metropolitano de Arequipa y Georreferenciación

6.1. Integración Cartográfica en Alta Definición

`MapaView` usa una imagen PNG de 1 536 × 1 536 px (`resources/mapa_arequipa.png`) como fondo de una `QGraphicsScene` cuya área es (−768, −768, 1 536, 1 536). La imagen se incrusta con `resources.qrc`, de modo que la aplicación funciona sin conexión. Los 20 nodos van desde el Parque Industrial (noroeste) hasta el Molino de Sabandía (sureste).

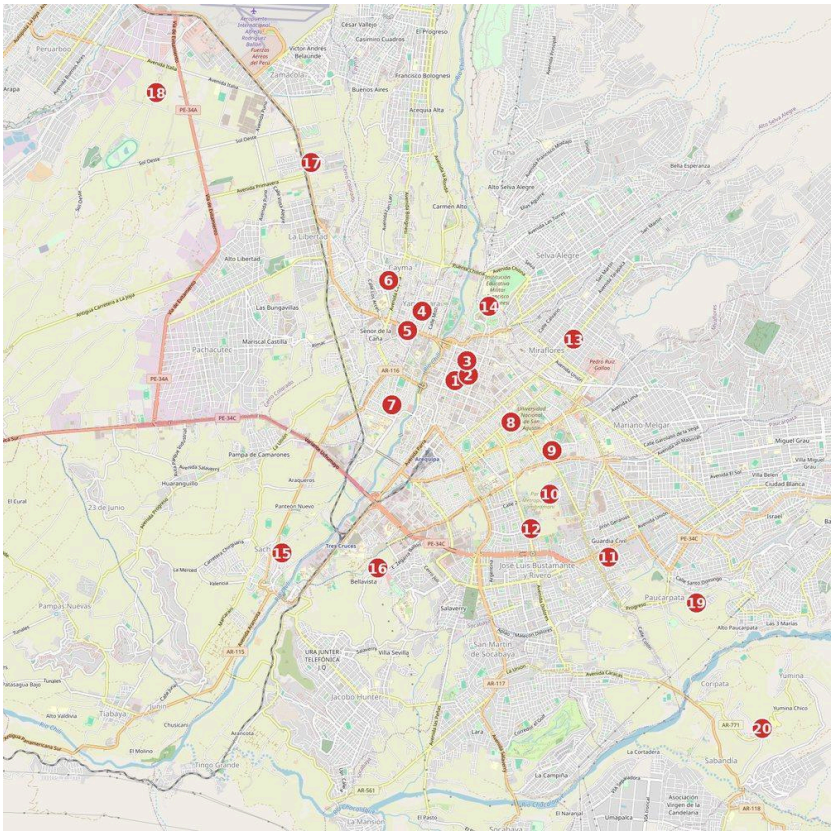


Figura 1. Mapa base de Arequipa con los 20 nodos logísticos (numeración de la Tabla 1, sección 6.3).

6.2. Proyección Matemática Web Mercator

Como el mapa mide 6 mosaicos de 256 px por lado, la correspondencia entre latitud/longitud y píxeles de la escena sigue la proyección Web Mercator estándar, donde `z` es el nivel de zoom del mosaico (14 según la memoria previa del proyecto) y `x_min`, `y_min` son los mosaicos de la esquina superior izquierda:

```
x_tile = ((lon + 180) / 360) * 2^z
y_tile = (1 - asinh(tan(rad(lat))) / pi) / 2 * 2^z
x_px   = (x_tile - x_min) * 256 - 768
y_px   = (y_tile - y_min) * 256 - 768
```

El repositorio guarda únicamente el resultado: las coordenadas de escena de los nodos son constantes en `MapaView::inicializarNodos()`, sin una función de conversión en tiempo de ejecución.

6.3. Catálogo de los 20 Nodos Logísticos Metropolitanos

Tabla 1. Nodos logísticos y coordenadas de escena (el distrito es referencial).

N.º	Nodo logístico	Distrito	Escena (X, Y)
1	Plaza de Armas	Arequipa (Cercado)	(70.7, -68.9)
2	Calle Mercaderes 120	Arequipa (Cercado)	(95.2, -78.6)
3	Calle San Francisco 204	Arequipa (Cercado)	(92.8, -105.3)
4	Mirador de Yanahuara	Yanahuara	(9.0, -197.6)
5	Av. Ejercito 450	Yanahuara / Cayma	(-17.8, -161.2)

N.º	Nodo logístico	Distrito	Escena (X, Y)
6	Av. Cayma 310	Cayma	(-52.8, -254.7)
7	Urb. Umacollo B-12	Yanahuara	(-47.0, -23.9)
8	UNSA Ingenierias (Av. Independencia)	Arequipa (Cercado)	(174.4, 8.9)
9	Av. Venezuela 800 (UNSA)	Arequipa (Cercado)	(250.1, 61.1)
10	Av. Lambramani 105	J. L. Bustamante y Rivero	(246.6, 142.5)
11	Mall Aventura Porongoche	Paucarpata	(355.0, 259.1)
12	Av. Dolores 890	J. L. Bustamante y Rivero	(211.7, 206.8)
13	Plaza Mayta Capac (Miraflores)	Miraflores	(290.9, -145.4)
14	Parque Selva Alegre	Alto Selva Alegre	(133.6, -206.1)
15	Mirador de Sachaca	Sachaca	(-250.9, 251.8)
16	Terminal Terrestre Arequipa	Jacobo Hunter	(-72.6, 279.7)
17	Cerro Colorado (Plaza Las Americas)	Cerro Colorado	(-196.1, -473.3)
18	Parque Industrial	Cerro Colorado / Río Seco	(-483.9, -603.2)
19	Plaza de Paucarpata	Paucarpata	(518.1, 344.1)
20	Molino de Sabandia	Sabandía	(640.4, 577.3)

Entre los nodos se dibuja una malla vial sencilla: se traza un segmento punteado entre cada par que esté a menos de 195 px.

6.4. Navegación e Interacción Tipo Google Maps

- **Arrastre:** `QGraphicsView::ScrollHandDrag` permite mover el mapa con el mouse.
- **Zoom:** la rueda del mouse escala la vista (×1.15 al acercar, ×0.85 al alejar).
- **Barra superior:** botones de acercar, alejar y «Centrar», que reencuadra el Centro Histórico.
- **Pines:** cada nodo se dibuja con un pin circular y una etiqueta con su nombre.

7. Calidad de Software y Verificación (Unit Testing)

Las pruebas usan macros propias (`ASSERT_TRUE`, `ASSERT_EQ`, `ASSERT_NEAR`) y se registran en CTest. Solo enlazan el modelo, sin Qt, así que corren rápido. Al compilar y ejecutar los tres ejecutables se obtuvo lo siguiente:

Suite	Qué verifica	Pruebas	Resultado
`test_dominio`	Polimorfismo de vehículos y paquetes, costos, capacidades, prioridades, repartidor	36	36 aprobadas
`test_generico`	`Cola<T>`, `Repositorio`, `Ruta`, functores y filtros con lambdas	31	31 aprobadas
`test_threading`	Simulador multihilo, productor/consumidor, atómicos, parada limpia, observer concurrente	8	8 aprobadas
Total		**75**	**0 fallos**

Queda pendiente cubrir con pruebas automáticas los controladores y las vistas Qt, que hoy solo se verifican de forma manual.

8. Trazabilidad de Trabajo y Control de Versiones (Git / GitHub)

El repositorio tiene 14 commits en la rama `main`, hechos entre el 5 y el 6 de noviembre de 2026, con mensajes al estilo Conventional Commits (`feat`, `fix`, `docs`, `chore`). Cada integrante trabajó en su módulo: Chipana 5 commits (4 con su usuario local y 1 desde la cuenta de GitHub), y Baca, Mejia y Poncel 3 cada uno.

Commit	Fecha	Autor	Mensaje
`1d2ecb5`	05/11	German Chipana	chore(setup): inicializacion de proyecto CMake, .gitignore y documentacion
`e6ee1ad`	05/11	L. Baca	feat(dominio): jerarquia de clases Vehiculo y Paquete, encapsulamiento y polimorfismo
`6a47a91`	05/11	G. Mejia	feat(generico): templates Cola<T>, Repositorio<T>, Ruta<CoordType> y gestion de memoria STL
`3eee1d0`	05/11	R. Poncel	feat(threading): multithreading, mutex, condicion, generador de eventos y patrones GoF
`609cacd`	05/11	German Chipana	feat(mvc-gui): controladores MVC, interfaz grafica en Qt6 y aplicacion principal
`3300475`	05/11	ChipanaGerman	Update team information in README
`a78e6c1`	06/11	German Chipana	fix(core): corregir sincronizacion de hilos en simulador, actualizacion reactiva de pedidos y mapa 2D
`617b1b3`	06/11	G. Mejia	fix(stats): corregir contador de completados y tasa de exito en estadisticas
`30aeed6`	06/11	R. Poncel	feat(sim): simular solo pedidos manuales y detener automaticamente al completar
`67f31c1`	06/11	G. Mejia	feat(core): asignacion de flota por tipo de vehiculo y despacho escalonado
`19983c7`	06/11	German Chipana	feat(gui): visualizacion sincronizada de flota y animacion pausada de rutas
`e7d8e7f`	06/11	L. Baca	feat(mapa): mapa urbano de Arequipa estilo Google Maps con 20 nodos
`115f730`	06/11	R. Poncel	docs: informe formal de avance del proyecto final SIGER
`4f81613`	06/11	L. Baca	docs: informe academico completo de avance en formato Word (.docx)

9. Conclusiones y Trabajo Futuro

9.1. Conclusiones

- SIGER cubre los seis temas del curso en una aplicación que ya corre completa: registro de pedidos, despacho concurrente y monitoreo visual.
- La separación MVC se cumple: el modelo no usa Qt y se prueba de forma aislada, mientras Observer y las señales de Qt conectan los hilos con la interfaz.

- La sincronización por variable de condición y atómicos evitó esperas activas y bloqueos en las pruebas de estrés realizadas.
- Las 75 pruebas unitarias pasan, aunque la interfaz gráfica aún depende de verificación manual.

9.2. Trabajo Futuro y Líneas de Expansión

- **Rutas reales:** reemplazar el desplazamiento directo entre nodos por un recorrido sobre la malla vial con Dijkstra o A*.
- **Tiempos coherentes con el modelo:** hoy la duración del viaje depende solo del tipo de vehículo; podría calcularse con la distancia y la velocidad de `calcularTiempoEntrega`.
- **Pedidos automáticos:** reactivar `GeneradorEventos` como opción de la interfaz para simular demanda continua.
- **Persistencia:** guardar el historial de pedidos y tiempos de entrega, por ejemplo con SQLite.
- **Pruebas de interfaz:** agregar pruebas para controladores y vistas con Qt Test.